Coulomb Matrix Based Molecular Energy Prediction Using Incremental PCA and Neural Network Regression

JSON-to-energy prediction pipeline

Abdullah Imran

21 June 2026

Contents

Abstract

This report presents a large-scale molecular machine learning pipeline for predicting molecular energies from JSON encoded molecular structures. Each molecule is represented by an energy target, a list of atoms, chemical element symbols, and Cartesian coordinates. The project converts this nested structure into a fixed-length numerical representation by mapping atom symbols to atomic numbers, computing pairwise interatomic distances, forming Coulomb matrices, flattening the matrices into dense feature vectors, standardizing the features, and reducing dimensionality using Incremental Principal Component Analysis. The final dataset contains approximately 171,137 molecules with a maximum molecule size of 121 atoms. This leads to 14,641 Coulomb features per molecule and approximately 2.51 billion dense feature values before dimensionality reduction. Standard PCA was not suitable at this scale due to memory and numerical constraints, so IncrementalPCA was implemented in explicit batches [4]. Several regressors were explored, including linear models, decision trees, K-nearest neighbors, random forests, gradient boosting, histogram gradient boosting, and multilayer perceptrons. The best final model was an MLPRegressor trained on the 1,400-component PCA representation with hidden layers of 1024, 512, 256, 128, 64, 64, 28, 64, and 16 neurons [5]. On a held-out 20% test set, it achieved an R2 score of 0.8465, RMSE of 13.8961, MAE of 9.0135, and MSE of 193.1011. These results demonstrate that a classical Coulomb matrix pipeline, when engineered carefully for memory and dimensionality reduction, can learn a useful surrogate relationship between molecular geometry and energy on consumer-scale hardware.

Keywords: molecular machine learning; Coulomb matrix; IncrementalPCA; dimensionality reduction; molecular energy prediction; MLPRegressor; quantum chemistry surrogate modeling.

1. Introduction

1.1 Scientific motivation

Molecular energy is a fundamental quantity in computational chemistry because it summarizes how favorable a molecular structure is under a given electronic structure model or reference calculation. In first-principles quantum chemistry, energies are obtained by solving or approximating the electronic Schroedinger equation,

$$\hat{H}\Psi = E\Psi,$$

where $\hat{H}$ is the Hamiltonian operator, Ψ is the electronic wavefunction, and E is the energy eigenvalue. Exact solutions are not feasible for general many-electron molecules, so practical calculations rely on approximations such as Hartree-Fock, density functional theory, perturbation theory, coupled cluster methods, and semi-empirical methods. These approaches provide the reference labels that make molecular machine learning possible, but they are computationally expensive when repeated across large chemical spaces.

Machine learning surrogate models are useful because they learn an approximate map [7],

$$f(\mathbf{X}) \approx E,$$

where $\mathbf{X}$ is a numerical representation of molecular structure and E is the reference energy. Once trained, such a model can make predictions much faster than repeating the underlying quantum chemical calculation. This project is therefore not a replacement for quantum mechanics. It is a supervised surrogate model that uses structural descriptors to approximate energies already present in the dataset.

1.2 Project objective

The objective of this project was to build a full end-to-end pipeline that can ingest JSON molecular data, generate physically motivated descriptors, handle large memory requirements, reduce dimensionality, train regression models, and support inference on new molecular JSON files. The project is technically significant because it does not only train a model on a precomputed feature table. It constructs the molecular representation itself from raw atom symbols and coordinates.

The final workflow is shown in Figure 1. It begins with nested JSON files and ends with predicted molecular energies from a saved preprocessing pipeline and trained neural network.

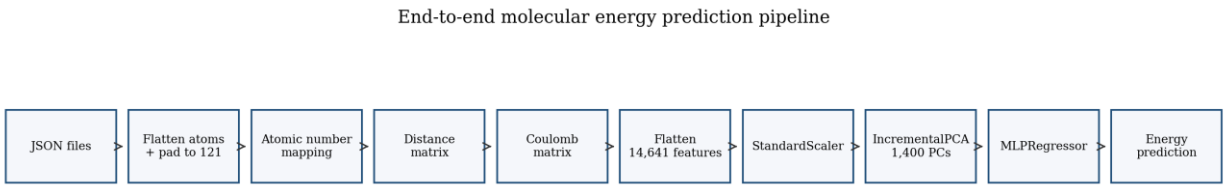


Figure 1. Overview of the final JSON-to-energy molecular machine learning workflow.

1.3 Main contributions

The main contributions of the project are as follows:

1. A reusable molecular preprocessing workflow from JSON files to Coulomb matrix descriptors.
2. A scalable handling strategy for approximately 171,137 molecules and 14,641 initial features per molecule.
3. A memory-aware conversion to float32 and the use of IncrementalPCA in batches.
4. A comparison of multiple machine learning regressors under realistic local hardware constraints.
5. A final MLPRegressor model achieving an R2 score of approximately 0.8465 on a held-out test set.
6. A saved inference pipeline that reuses the training scaler, PCA object, maximum atom count, and final model.

2. Dataset and JSON structure

2.1 Molecular JSON schema

The input data consist of multiple JSON files obtained from the Kaggle Predict Molecular Properties dataset [2]. Each file contains a list of molecule objects. Each molecule contains an energy field named En and an atom list named atoms. Each atom stores an element symbol under type and a three-dimensional Cartesian coordinate vector under xyz. The effective schema is:

```
[
  {
    "En": <molecular_energy>,
    "atoms": [
      {"type": "C", "xyz": [x0, y0, z0]},
      {"type": "H", "xyz": [x1, y1, z1]},
      {"type": "O", "xyz": [x2, y2, z2]}
    ]
```

}
]
This structure contains the minimum information needed for a Coulomb matrix representation: nuclear identities and three-dimensional coordinates. The target variable is the energy field En, which the dataset describes as molecular energy calculated using a force-field method [2].

2.2 Flattened molecular table

Each molecule was converted into a fixed-width row. The maximum molecule size observed during training was 121 atoms, so each row reserves 121 atom slots. Each atom slot contributes four columns:

1. atom_i_type
2. atom_i_x
3. atom_i_y
4. atom_i_z

The approximate flattened dataframe shape was:

$$(171137,485),$$

where 485 columns correspond to one energy column plus 121×4 atom-related columns. Smaller molecules are padded with empty atom slots so that all molecules can be represented by the same number of columns.

2.3 Atomic number mapping

The chemical symbol column is mapped to atomic number Z. For example, hydrogen maps to 1, carbon maps to 6, nitrogen maps to 7, and oxygen maps to 8. Padding atoms are mapped to $Z = 0$. This is essential because the Coulomb matrix is defined using nuclear charges, not categorical symbols.

2.4 Atom-count metadata

The number of atoms per molecule was recorded during preprocessing. This metadata was useful for dataset characterization and for confirming that the maximum atom count was 121. However, it was not included as an explicit model feature because molecule size is already implicitly encoded in the Coulomb matrix through the number of non-zero rows, columns, and pairwise interactions.

3. Coulomb matrix representation

3.1 Mathematical definition

For a molecule with atoms indexed by i and j, atomic numbers Z_i and Z_j, and Cartesian coordinate vectors $\mathbf{R}_i$ and $\mathbf{R}_j$, the Coulomb matrix C is defined as:

$$C_{ij} = \begin{cases} \dfrac{Z_i Z_j}{\| \mathbf{R}_i - \mathbf{R}_j \|}, & i \neq j, \\ 0.5 Z_i^{2.4}, & i = j. \end{cases}$$

The off-diagonal elements approximate pairwise nuclear Coulomb repulsion. The diagonal elements encode isolated atom contributions using the empirical form popularized in molecular machine learning work using Coulomb matrix descriptors [1].

3.2 Distance matrix calculation

The Euclidean interatomic distance is:

$$R_{ij} = \| \mathbf{R}_i - \mathbf{R}_j \|_2.$$

For computational efficiency, pairwise distances were computed using the Gram matrix identity:

$$\| \mathbf{R}_i - \mathbf{R}_j \|^2 = \| \mathbf{R}_i \|^2 + \| \mathbf{R}_j \|^2 - 2\mathbf{R}_i \cdot \mathbf{R}_j.$$

This avoids explicit Python loops over atom pairs and allows NumPy to perform the major operations using vectorized routines.

3.3 Invariance properties and limitations

The Coulomb matrix depends on interatomic distances and nuclear charges. It is therefore invariant to rigid translation and rotation of the molecule. However, the flattened matrix is sensitive to atom ordering unless the matrix is sorted, randomized, or converted into a more permutation-invariant representation. This is a known caveat of raw Coulomb matrices [1,8]. In this project, the JSON atom ordering was preserved, and the downstream PCA and MLP model learned from that representation.

3.4 Feature dimensionality

The maximum molecule size of 121 atoms implies a fixed Coulomb matrix of shape:

$$121 \times 121.$$

The flattened feature vector therefore contains:

$$121^2 = 14641$$

features per molecule. Figure 2 summarizes how the feature count changes across the pipeline.

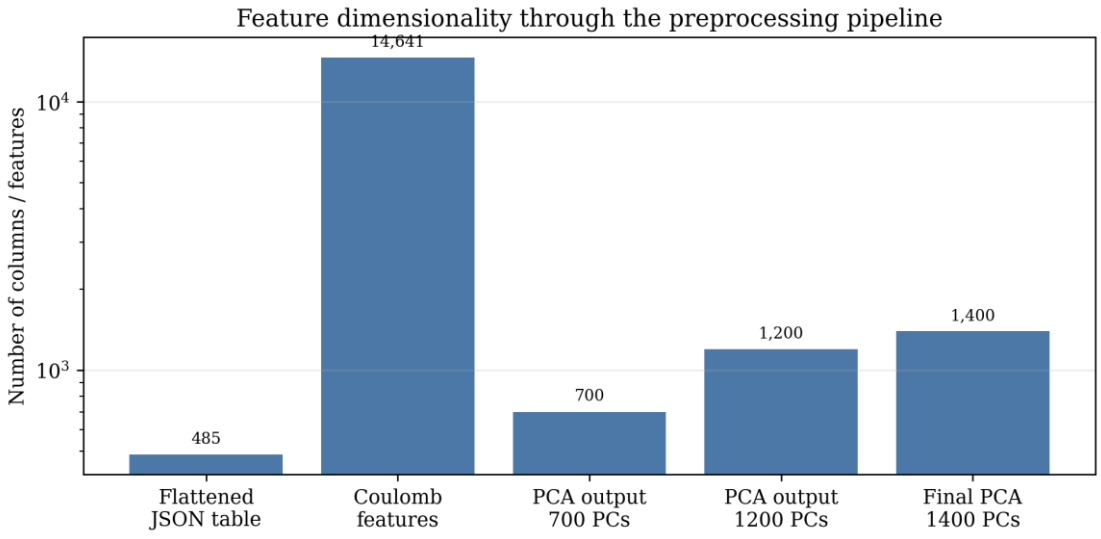


Figure 2. Feature dimensionality through the preprocessing pipeline. The raw Coulomb representation is reduced from 14,641 features to the final 1,400 PCA components.

4. Data scale, memory constraints, and numerical precision

4.1 Dense matrix size

The full Coulomb feature matrix contains:

$$171137 \times 14641 = 2505616817$$

dense values. If stored as float64, this single dense matrix requires approximately:

$$2505616817 \times 8 \approx 18.67 \text{ GiB}.$$

This estimate matches the observed memory failure during development. The actual memory burden is even higher because the workflow also creates intermediate coordinate tensors, Gram matrices, distance matrices, scaled arrays, pandas objects, and PCA outputs.

4.2 Precision choice

The project converted major arrays to float32. This reduces the storage cost per value from 8 bytes to 4 bytes:

$$\text{float32 memory} \approx \frac{1}{2}\,\text{float64 memory}.$$

Figure 3 shows the approximate memory reduction for one dense Coulomb feature matrix.

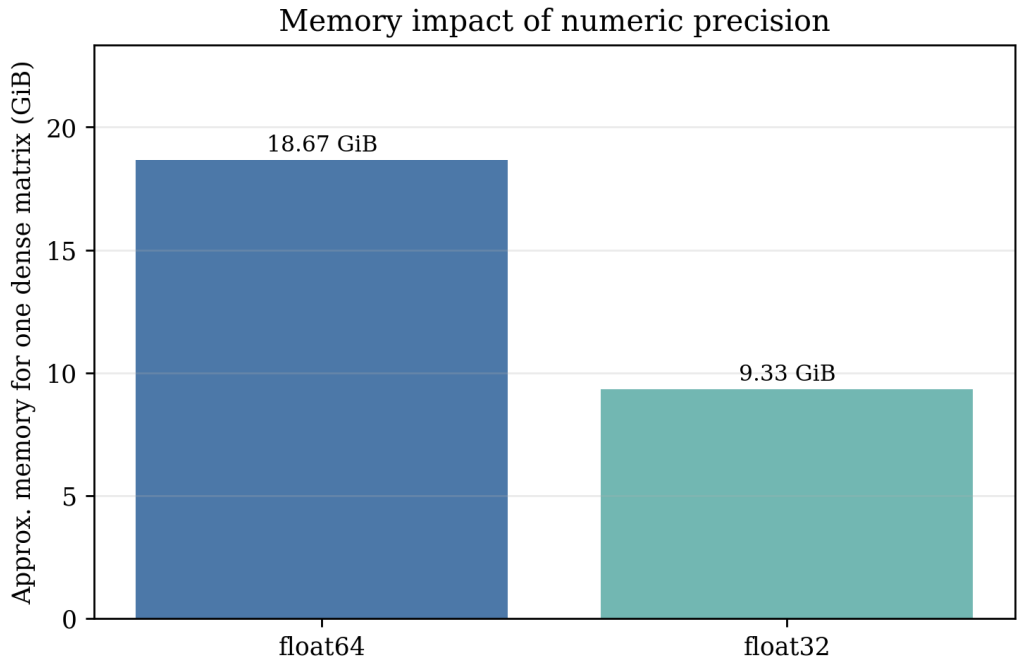


Figure 3. Memory impact of numeric precision. Converting from float64 to float32 approximately halves the memory needed for one dense Coulomb matrix.

Float32 was appropriate because the downstream task is statistical regression and the target standard deviation is approximately 35 energy units. Float16 was not used because its reduced precision could destabilize distance calculations, division by small distances, and PCA.

4.3 Storage reduction after PCA

The final PCA matrix has shape:

$$171137 \times 1400.$$

In float32, this is approximately 0.89 GiB, much smaller than the 9.33 GiB raw float32 Coulomb matrix and 18.67 GiB float64 matrix. Figure 4 summarizes the storage reduction from precision control and PCA compression.

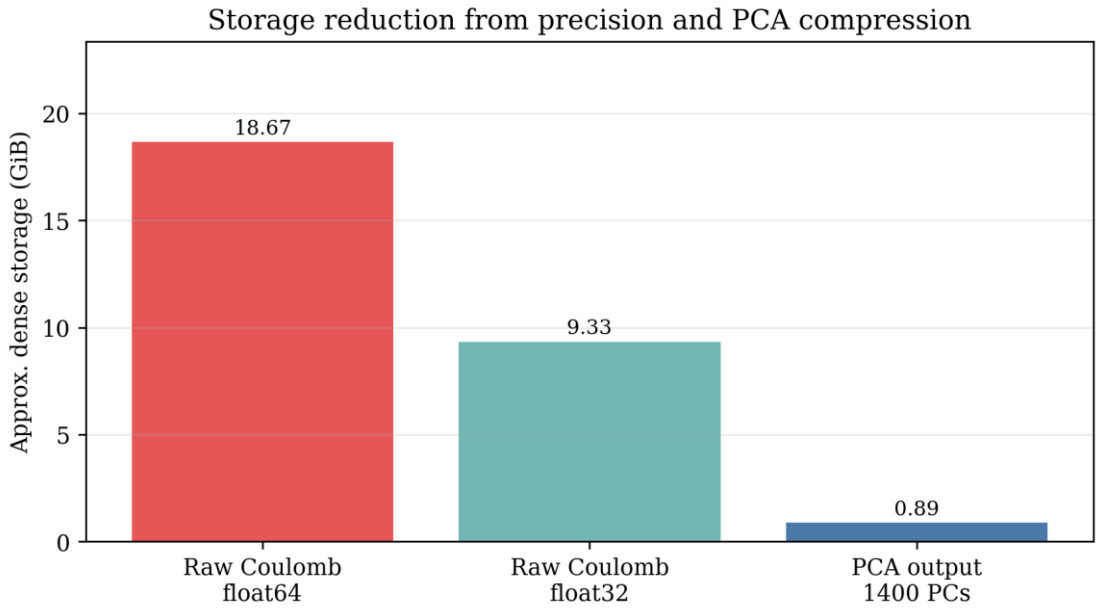


Figure 4. Approximate storage reduction from float32 precision and PCA compression.

4.4 Computational cost and runtime observations

Several runtime observations shaped the final modeling decisions. K-nearest neighbors became expensive because prediction requires distance comparisons against many training examples. Random forests were promising but slow at this dataset size and feature dimensionality. Histogram gradient boosting was much faster and produced a strong intermediate result. The MLP became viable after PCA compression reduced the feature dimension to 1,400.

An observed MLP timing test on a 50,000 molecule subset showed 30 iterations in approximately 2 minutes and 33 seconds, or about 5.1 seconds per iteration. Scaling roughly by sample count from 50,000 to 171,137 gives an estimated full-dataset iteration cost of approximately 17 seconds per iteration under similar conditions. This estimate explains why the larger MLP was feasible.

5. Incremental PCA

5.1 Why standard PCA was not suitable

Standard PCA requires a large decomposition of the full dense feature matrix or an equivalent operation that is not practical at the observed scale. The project encountered an integer overflow or memory-related LAPACK failure when attempting standard PCA. This is expected when working with a matrix containing approximately 2.51 billion dense values.

5.2 Batch-wise IncrementalPCA strategy

IncrementalPCA was adopted because it allows PCA to be fit in batches and is designed for memory-efficient dimensionality reduction on large datasets [4]. The practical algorithm used two passes. The first pass fitted PCA components using partial_fit. The second pass transformed batches into the reduced PCA space.

```
pca = IncrementalPCA(n_components=1400, batch_size=1400)

for start in range(0, x_scaled.shape[0], 1400):
    end = min(x_scaled.shape[0], start + 1400)
    pca.partial_fit(x_scaled[start:end])

x_pca = np.zeros((x_scaled.shape[0], 1400), dtype=np.float32)

for start in range(0, x_scaled.shape[0], 1400):
    end = min(start + 1400, x_scaled.shape[0])
    x_pca[start:end] = pca.transform(x_scaled[start:end]).astype(np.float32)
```

The important implementation point is that fit_transform was avoided on the full scaled matrix. The pipeline used partial_fit and transform explicitly in batches.

5.3 Variance retention

The project compared multiple PCA component counts. The observed variance retention values were approximately:

PCA components	Cumulative explained variance
700	90.14%
1200	93.40%
1400	94.90%

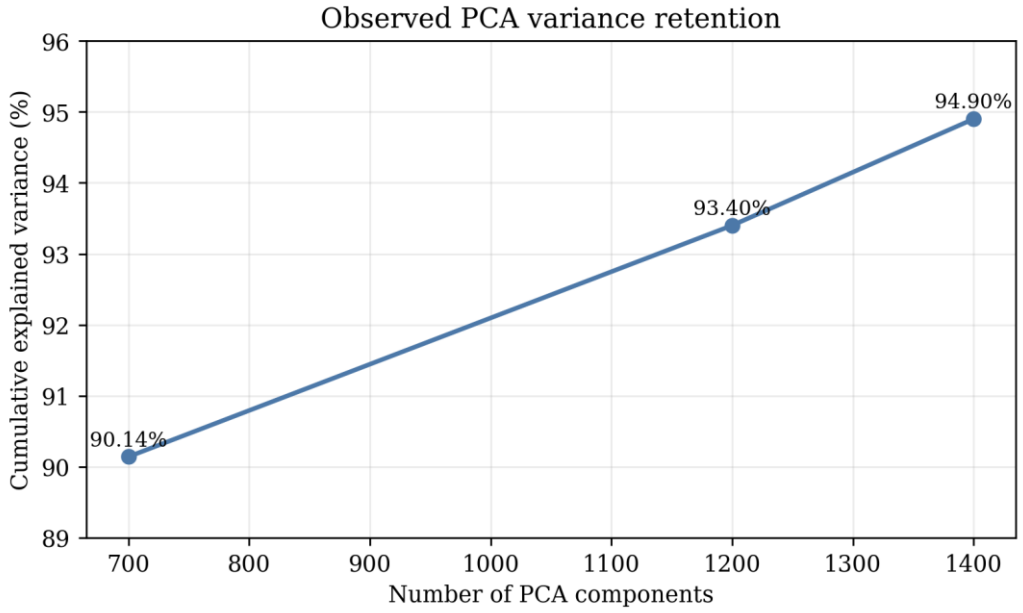


Figure 5. Observed PCA variance retention across tested component counts.

The curve shows diminishing returns. Moving from 700 to 1,400 components doubles the PCA feature count but recovers only about 4.8 additional percentage points of variance. The final choice of 1,400 components was therefore justified by the improvement in model performance, not by variance retention alone.

5.4 Why the Kaiser criterion was not used

The Kaiser criterion keeps principal components whose eigenvalues exceed 1. It is useful in some exploratory factor analysis settings with standardized variables and moderate dimensionality. It was not used as a final decision rule here because the original feature space contains 14,641 Coulomb entries. In such high-dimensional data, the criterion is not as meaningful as validation performance and computational feasibility. The project therefore used explained variance, model performance, and runtime constraints as the practical selection criteria.

6. Exploratory data analysis

6.1 Energy target statistics

The energy target had the following approximate statistics during development:

Statistic	Value
Minimum energy	-99
Maximum energy	460
Standard deviation	35.24

The residual distributions from trained models were approximately centered around zero. This made MAE, RMSE, and R2 more interpretable than percentage-based metrics.

6.2 Atom-count distribution

The dataset contains molecules from 1 atom up to 121 atoms. Most molecules are concentrated in the 20 to 60 atom range, with a right tail toward larger molecules. Figure 6 shows the atom-count distribution.

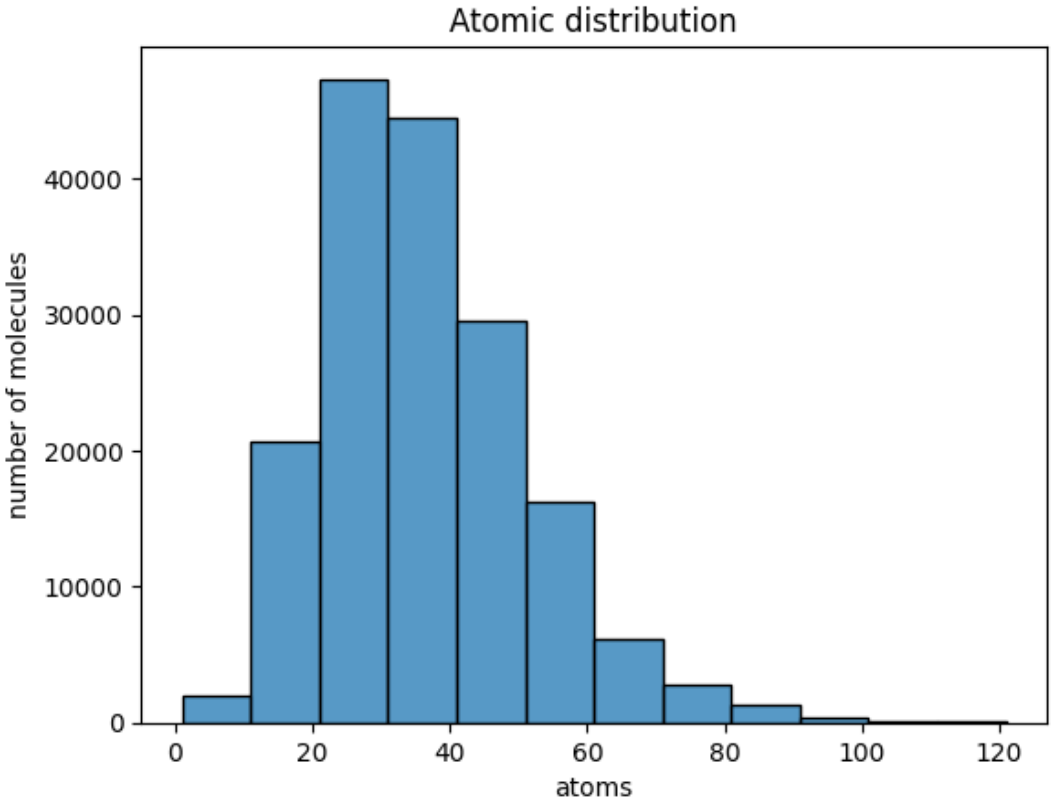


Figure 6. Atom-count distribution in the training data. Most molecules fall between approximately 20 and 60 atoms, while the 121-atom maximum lies in the tail.

The fact that most molecules are smaller than 121 atoms means the padded Coulomb representation contains many zero-like or low-information regions for smaller molecules. This helps explain why PCA is effective.

6.3 Removal of atom-count error subgroup analysis

Prediction-error statistics by atom-count ranges were explored. However, subgroup R2 values were not useful as a final narrative because R2 is sensitive to the variance of the target within each subgroup. A subgroup can show lower R2 even when absolute errors are reasonable if the subgroup target variance is narrow. The final report therefore keeps atom count as dataset metadata rather than as a major evaluation axis.

7. Model development and selection

7.1 Evaluation protocol

The project used an 80/20 train-test split:

$$\text{training set} = 80\%, \qquad \text{test set} = 20\%.$$

The split used random_state = 10 for reproducibility. Models were evaluated using mean squared error, root mean squared error, mean absolute error, and R2:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2,$$

$$\text{RMSE} = \sqrt{\text{MSE}},$$

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|,$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}.$$

MAPE was not emphasized because the target can be close to zero or negative, which makes percentage errors unstable.

7.2 Early baseline models

The early modeling phase tested Ridge, Lasso, Decision Tree, KNN, Gradient Boosting, Random Forest, and small MLP configurations. These tests were useful for diagnosing whether the Coulomb-PCA representation contained learnable signal. The early baseline results should not all be interpreted as final comparable benchmarks because they were often run on earlier PCA configurations and exploratory settings.

The broad conclusions were:

- Linear models were fast but underfit the nonlinear structure.
- A single decision tree had limited generalization.
- KNN was computationally unattractive at full scale.
- Random forest was promising but too slow to be practical on the available machine.

- Histogram gradient boosting was a strong and efficient tabular baseline.
- MLPRegressor achieved the best final performance after increasing PCA dimensionality and model capacity.

7.3 Histogram gradient boosting baseline

HistGradientBoostingRegressor was considered because histogram-based gradient boosting is designed to scale better on large tabular datasets than classical gradient boosting [6]. It achieved a strong comparison result with approximately:

Metric	HistGradientBoostingRegressor
MSE	234.8760
RMSE	15.3257
MAE	10.3982
R2	0.8133

This result demonstrated that the PCA representation contained strong predictive signal even before the final neural network model.

7.4 MLP model family

The first strong MLP used the architecture:

```
hidden_layer_sizes = (512, 256, 128, 64, 64, 28, 64, 16)
learning_rate_init = 0.0008
```

It achieved approximately:

Metric	MLP with 512 first layer
MSE	212.6876
RMSE	14.5838
MAE	9.5677
R2	0.8309

An additional experiment increased the first hidden layer to 1,024 neurons and used the architecture:

```
hidden_layer_sizes = (1024, 512, 256, 128, 64, 64, 28, 64, 16)
learning_rate_init = 0.00025
```

This model achieved the best result:

Metric	Final MLP with 1024 first layer
MSE	193.1011
RMSE	13.8961
MAE	9.0135
R2	0.8465

Figure 7 shows the final MLP architecture.

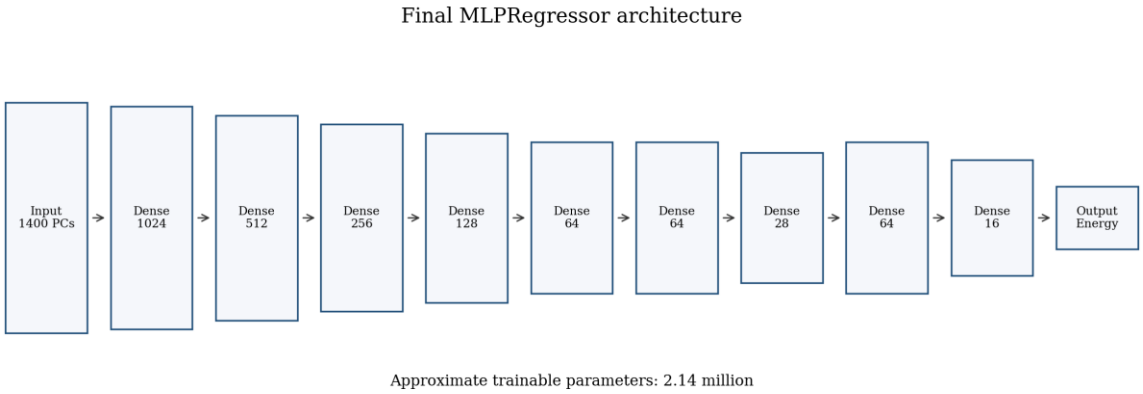


Figure 7. Final MLPRegressor architecture. The model maps the 1,400-component PCA feature vector to a scalar energy prediction.

7.5 Computational caveats with other models

Model selection was not based only on accuracy. It also depended on feasibility. KNN required expensive distance operations at prediction time. Random forests were promising but the repeated training cost became too high. Large MLPs were also costly, but PCA compression made them feasible enough for overnight training.

8. Final results

8.1 Final model selection

The final selected model is:

```
MLPRegressor(
   hidden_layer_sizes=(1024,512,256,128,64,64,28,64,16),
   learning_rate_init=0.00025,
   verbose=1
)
```

It was trained on the 1,400-component PCA representation, which retained approximately 94.9% of the variance in the standardized Coulomb features.

8.2 Final model comparison

The final-stage comparison is summarized in Table 1 and Figures 8 and 9.

Model	PCA setting	MSE	RMSE	MAE	R2
HistGradientBoostingRegressor	1200/1400 comparison stage	234.8760	15.3257	10.3982	0.8133
MLP, 512 first layer	1400 PCs	212.6876	14.5838	9.5677	0.8309
Final MLP, 1024 first layer	1400 PCs	193.1011	13.8961	9.0135	0.8465

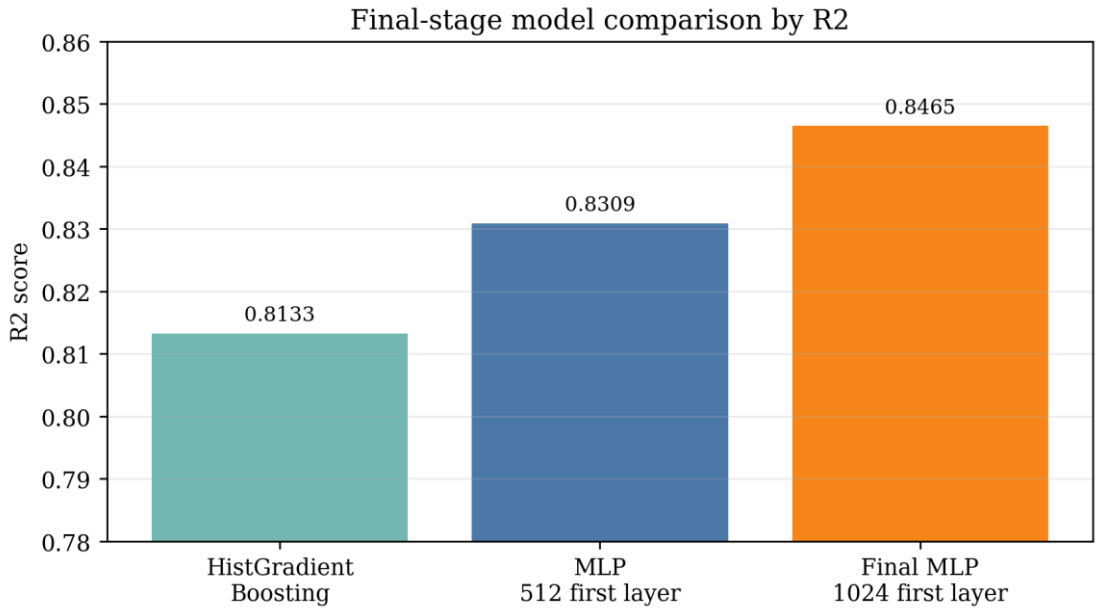


Figure 8. Final-stage model comparison by R2 score.

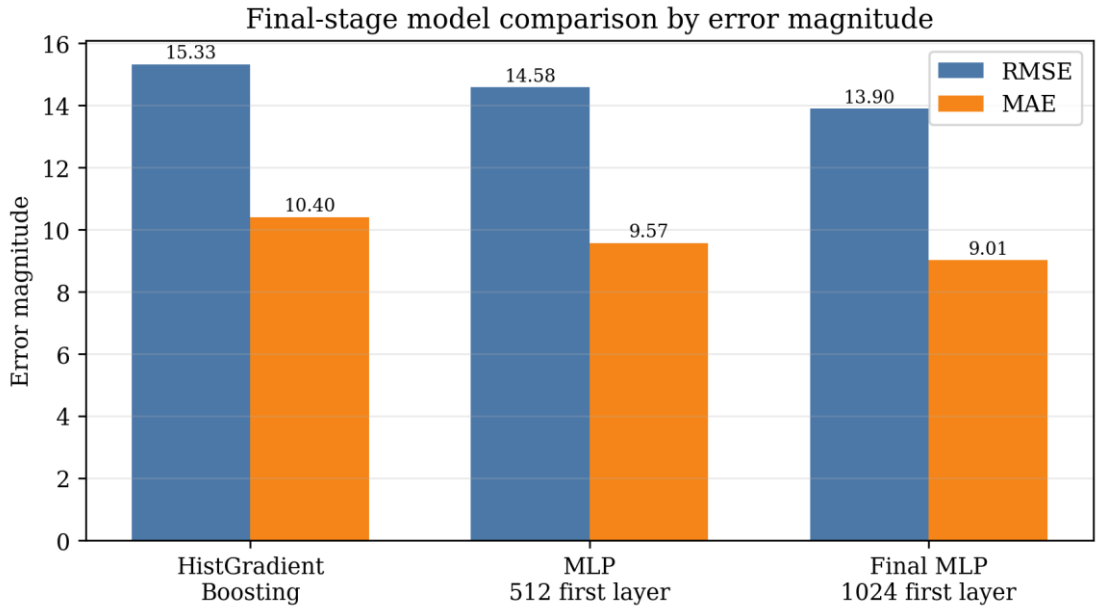


Figure 9. Final-stage model comparison by RMSE and MAE.

8.3 Interpretation of the achieved metrics

The final R2 score of 0.8465 means that the model explains approximately 84.65% of the variance in held-out molecular energies. Given the target standard deviation of approximately 35.24, the final RMSE of 13.90 is approximately:

$$\frac{13.90}{35.24} \approx 0.39$$

standard deviations of the target. The final MAE of 9.01 is approximately:

$$\frac{9.01}{35.24} \approx 0.26$$

standard deviations of the target. This is a useful predictive model under the constraints of a classical representation, PCA compression, and scikit-learn modeling on local hardware.

The improvement from the 512-layer MLP to the 1024-layer MLP is also meaningful. The R2 increased from approximately 0.8309 to 0.8465, while RMSE decreased from 14.58 to 13.90 and MAE decreased from 9.57 to 9.01. This suggests that the 1,400-component representation still contained learnable nonlinear signal that a larger neural network could exploit.

9. Inference pipeline

9.1 Saved artifacts

The final workflow relies on saved artifacts:

1. train_data_pca1400.joblib, containing PCA-transformed features and energy targets.
2. coloumb_pca_pipeline1400.joblib, containing the fitted scaler, fitted IncrementalPCA object, maximum atom count, and atom-count metadata.
3. mlp1024_512_256_128_64_64_28_64_16, containing the trained final MLP model.

9.2 Inference steps

A new molecular JSON file must pass through the same transformation sequence as the training data:

```
pipeline = joblib.load("coloumb_pca_pipeline1400.joblib")
model = joblib.load("mlp1024_512_256_128_64_64_28_64_16")

sc = pipeline["scaler"]
pca = pipeline["pca"]
max_atoms = pipeline["max_atoms"]

# JSON -> Coulomb matrix -> flatten
x_scaled = sc.transform(x)
x_test = pca.transform(x_scaled)
y_pred = model.predict(x_test)
```

The key rule is that inference must use transform, not fit_transform. Re-fitting the scaler or PCA during inference would destroy compatibility with the training feature space.

9.3 Domain constraints

The inference pipeline assumes that no molecule exceeds 121 atoms. This is not merely a code choice. It is required because the scaler and PCA were fitted on 14,641-dimensional Coulomb vectors. Any molecule with more than 121 atoms would generate a larger matrix and require retraining the representation pipeline.

10. Discussion

10.1 Relevance of the project

This project is relevant because it demonstrates a complete surrogate modeling workflow. The final model does not simply fit a ready-made CSV file. It constructs a molecular representation from raw atom identities and coordinates, solves substantial memory issues, performs scalable PCA, compares model families, and produces

a reusable inference path. This is the type of engineering required before molecular machine learning can be applied to large chemical datasets.

10.2 Scientific interpretation

The Coulomb matrix encodes a physically meaningful approximation of the nuclear framework. The model does not explicitly solve electronic structure equations, but it learns statistical relationships between nuclear positions, nuclear charges, and the target energy labels. The achieved R2 of 0.8465 shows that this representation captures a substantial amount of the energy-relevant structure. However, it also shows that the representation is not complete. About 15.35% of variance remains unexplained by the final model on the test set.

10.3 Limitations

The main limitations are:

1. Atom-order sensitivity of the raw Coulomb matrix [8].
2. Loss of information from PCA compression.
3. Lack of explicit bond topology or chemical graph information.
4. Dependence on the chemical distribution of the training JSON files.
5. Computational limits that prevented exhaustive model and representation search.
6. Limited interpretability of PCA-transformed features.

10.4 Future work

Future versions could test sorted Coulomb matrices, Bag of Bonds, many-body descriptors, SOAP descriptors, atom-centered symmetry functions, molecular fingerprints, or graph neural networks [9]. These representations may improve accuracy by encoding chemical invariance and local structural environments more directly. However, they also require additional implementation effort and computational resources.

11. Conclusion

This project developed a complete molecular energy prediction pipeline from JSON molecular structures to deployable machine learning inference. The final system processes approximately 171,137 molecules, constructs 121 by 121 Coulomb matrices, compresses 14,641 raw features to 1,400 PCA components using IncrementalPCA, and trains a final MLPRegressor that achieves R2 = 0.8465, RMSE = 13.8961, MAE = 9.0135, and MSE = 193.1011 on the held-out test set. The result is strong for a classical Coulomb matrix plus PCA workflow implemented under local hardware constraints. The project successfully demonstrates how molecular structure can be converted into a scalable statistical surrogate for energy prediction while also identifying clear directions for future improvement.

References

[1] Rupp, M.; Tkatchenko, A.; Müller, K.-R.; von Lilienfeld, O. A. Fast and Accurate Modeling of Molecular Atomization Energies with Machine Learning. Physical Review Letters 108, 058301 (2012). DOI: 10.1103/PhysRevLett.108.058301.

[2] BurakH (burakhmmtgl). Predict Molecular Properties. Kaggle Dataset. Available at: https://www.kaggle.com/datasets/burakhmmtgl/predict-molecular-properties/data. Accessed 21 June 2026.

[3] Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; Vanderplas, J.; Passos, A.; Cournapeau, D.; Brucher, M.; Perrot, M.; Duchesnay, É. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, 2825-2830 (2011).

[4] scikit-learn developers. sklearn.decomposition.IncrementalPCA. scikit-learn API Reference. Accessed 21 June 2026.

[5] scikit-learn developers. sklearn.neural_network.MLPRegressor. scikit-learn API Reference. Accessed 21 June 2026.

[6] scikit-learn developers. sklearn.ensemble.HistGradientBoostingRegressor. scikit-learn API Reference. Accessed 21 June 2026.

[7] Hansen, K.; Biegler, F.; Ramakrishnan, R.; Pronobis, W.; von Lilienfeld, O. A.; Müller, K.-R.; Tkatchenko, A. Machine Learning Predictions of Molecular Properties: Accurate Many-Body Potentials and Nonlocality in Chemical Space. Journal of Physical Chemistry Letters 6, 2326-2331 (2015). DOI: 10.1021/acs.jpclett.5b00831.

[8] Huang, B.; von Lilienfeld, O. A. Communication: Understanding Molecular Representations in Machine Learning: The Role of Uniqueness and Target Similarity. Journal of Chemical Physics 145, 161102 (2016). DOI: 10.1063/1.4964627.

[9] Raghunathan, S.; Priyakumar, U. D. Molecular Representations for Machine Learning Applications in Chemistry. International Journal of Quantum Chemistry 122, e26870 (2022). DOI: 10.1002/qua.26870.

Appendix A. Representative implementation blocks

A.1 Correct metric calculation

```
r2 = r2_score(actual, predicted)
mse = mean_squared_error(actual, predicted)
mae = mean_absolute_error(actual, predicted)
rmse = root_mean_squared_error(actual, predicted)
```

A.2 Final model training block

```
x, y = joblib.load("train_data_pca1400.joblib")
original_set = joblib.load("coloumb_pca_pipeline1400.joblib")
n_atoms = original_set["n_atoms"]

x_train, x_test, y_train, y_test, n_atoms_train, n_atoms_test = train_test_split(
    x, y, n_atoms, test_size=0.2, random_state=10
)

mlp1024 = MLPRegressor(
    hidden_layer_sizes=(1024,512,256,128,64,64,28,64,16),
    learning_rate_init=0.00025,
    verbose=1
)

mlp1024.fit(x_train, y_train)
predicted = mlp1024.predict(x_test)
```

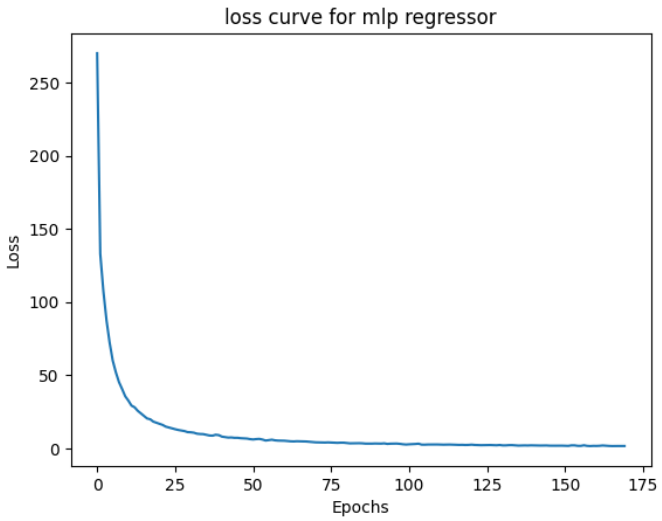
A.3 IncrementalPCA block

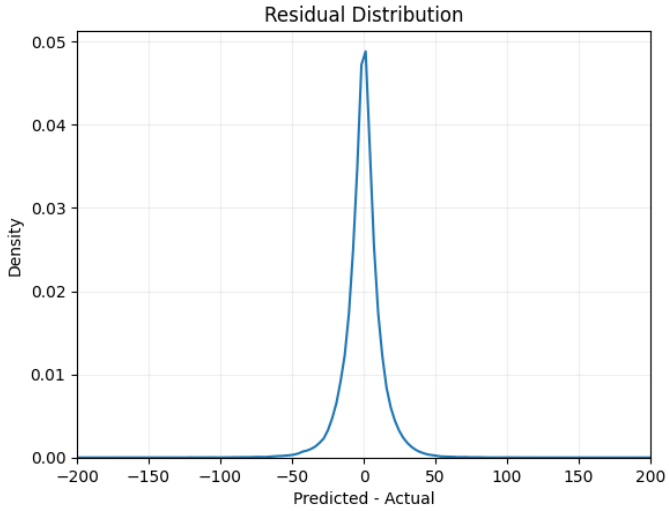
```
pca = IncrementalPCA(n_components=1400, batch_size=1000)

for start in range(0, x_scaled.shape[0], 1000):
    end = min(x_scaled.shape[0], start + 1000)
    pca.partial_fit(x_scaled[start:end])

x_pca = np.zeros((x_scaled.shape[0], 1400), dtype=np.float32)

for start in range(0, x_scaled.shape[0], 1000):
    end = min(start + 1000, x_scaled.shape[0])
    x_pca[start:end] = pca.transform(x_scaled[start:end]).astype(np.float32)
```

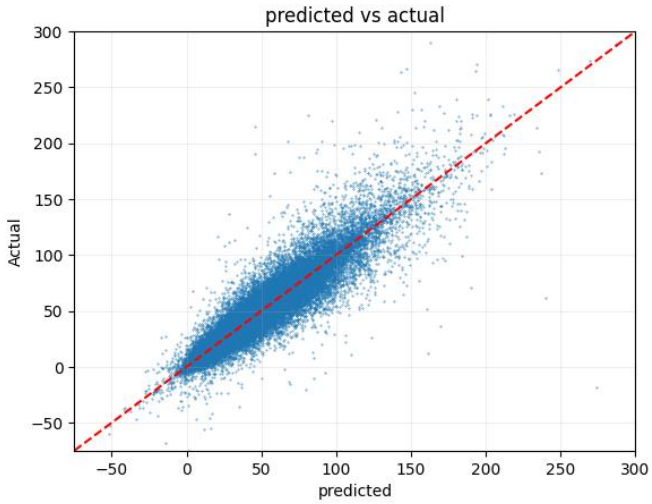
Appendix B. Notebook diagnostic plots

This appendix reproduces the principal diagnostic plots generated directly from the notebook during model development. The figures include representative MLP loss curves, residual density plots, and predicted-versus-actual scatter plots for the major experimental stages discussed in the report. They are included here to preserve the original exploratory workflow and to document how the final modeling decisions were reached.

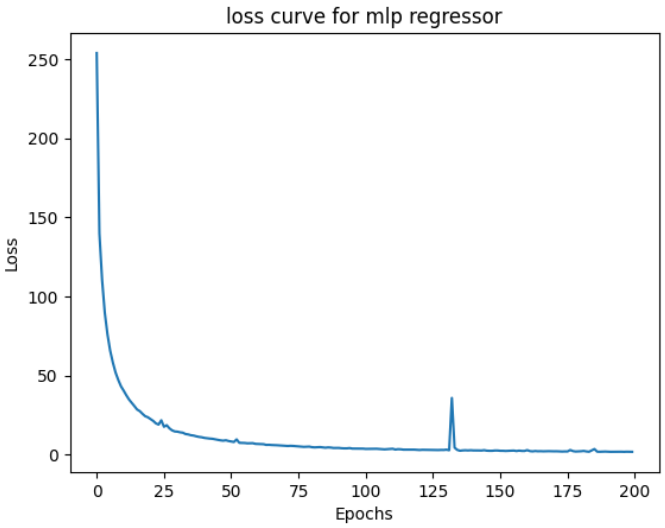
1. MLP (1024 neurons)

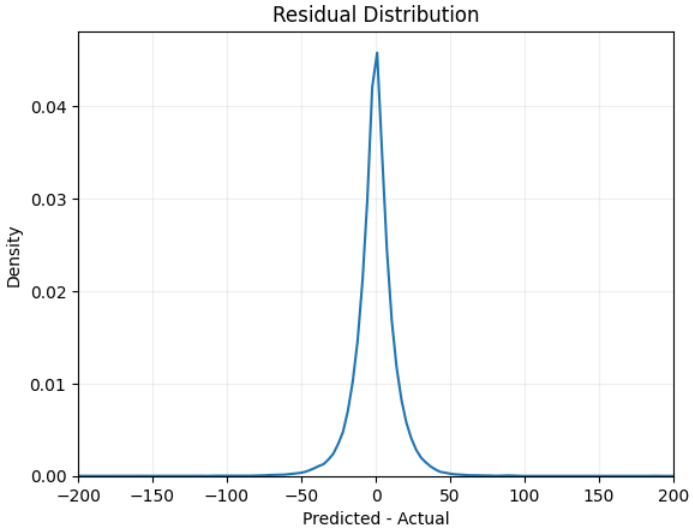


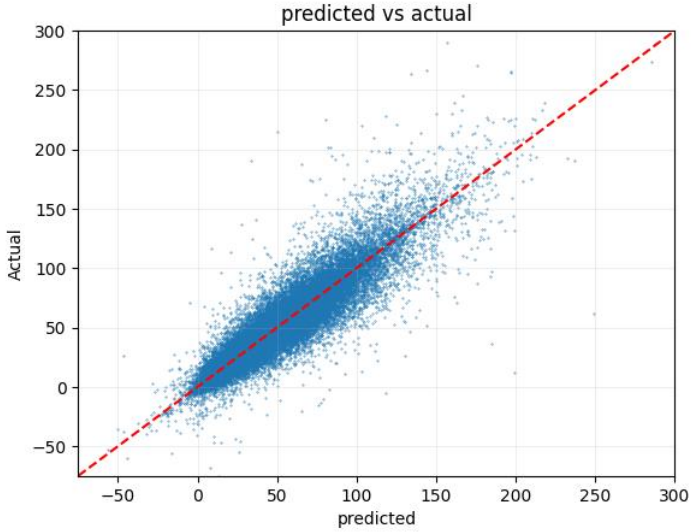




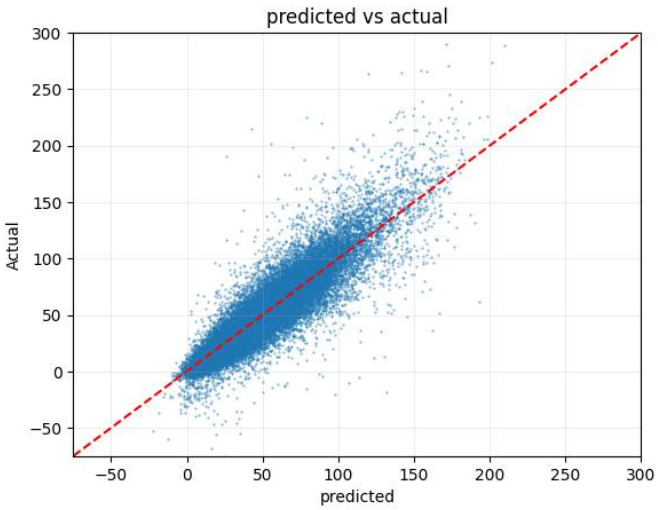
2. MLP (524 neurons)

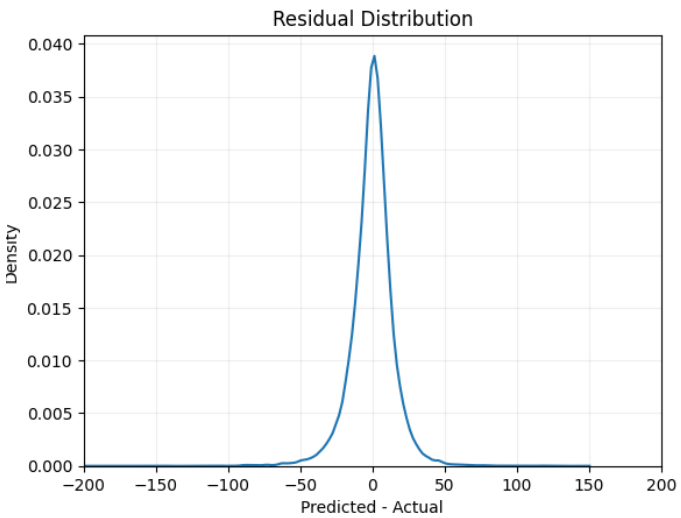






3. HistGradientBoosting





4. Average energy by molecule sizes